Fundamentos de Aprendizagem Automática

Class 07 — Ensemble Models

Mário Antunes

Universidade de Aveiro

2026

One Model Is a Guess

Where We Left Off

linear, probabilistic
Classes 03–04

margin, network
Class 05

tree, neighbours
Class 06

Class 07: many models, *one* decision

- Until now: **one** learner, tuned
- Today: several learners **decide together**

Roadmap of the Class

Part	Question	Answer
One model is a guess	do good models agree?	no: the gallery
Why combining helps	when does a crowd win?	independent, diverse errors
Voting	how to combine?	hard, soft, weighted vote
Bagging, forests	how to cut variance?	bootstrap, average
Boosting	how to cut bias?	fix the errors in turn
Stacking	how to learn the mix?	a meta-learner
Using ensembles	which one, at what cost?	a protocol and a table

Notation N7: Members

Each symbol is new in this class; earlier symbols keep their meaning.

symbol	meaning	shape	example
B	number of members	scalar	$B = 100$ trees
b	index of a member or round	scalar	$b = 1, \dots, B$
$\hat{f}_b$	member b	function	one tree
$\hat{f}_{\text{ens}}$	the ensemble	function	the vote
$\hat{p}_b(k \mid x)$	probability member b gives to class k	scalar	0.8
η	shrinkage (the learning rate of Class 02)	scalar	0.1

Notation N7b: Voters, Variance, Weights

symbol	meaning	shape	example
γ	accuracy of one voter (p in the plot)	scalar	0.6
a_B	accuracy of the majority of B voters	scalar	$a_3 = 0.648$
v, V_B	variance of one member, of the mean of B	scalar	$V_5 = 0.680$
ρ	correlation of two members' errors	scalar	$\rho = 0.6$
ξ	share of examples left out of a bootstrap sample	scalar	0.368
$w_i^{(b)}$	weight of example i in round b	scalar	$1/n$ at the start
err_b	weighted error of member b	scalar	0.20
β_b	weight of member b in the vote	scalar	0.693
E_b, Z_b	exponential loss of member b, weight normaliser	scalar	0.800
n	number of examples (Class 01)	scalar	210
x_i	example i (Class 01)	vector	one moons point
y_i	targets (Class 01)	scalar	$+1$ or -1

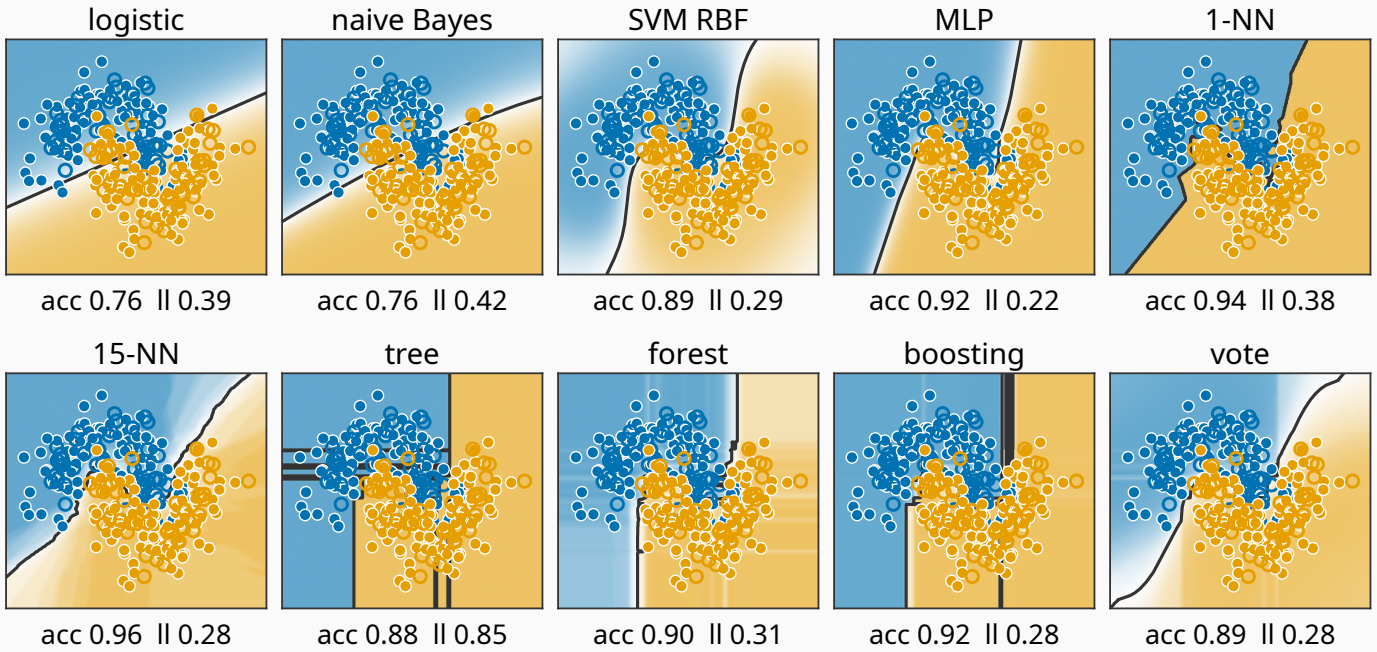
The Moons: One Running Example

Two interleaved half-circles with noise: easy to draw, hard to separate with a line.

- $n = 300$ points, noise 0.3, stratified 70/30 split
- Train 210 points (105 per class), test 90 (45 per class): **one test point is 0.011**
- Best possible accuracy (k-NN on 40 000 moons): about **0.92**
- Every number in this class comes from the demo notebooks (E1 to E5)

Ten Models, One Task

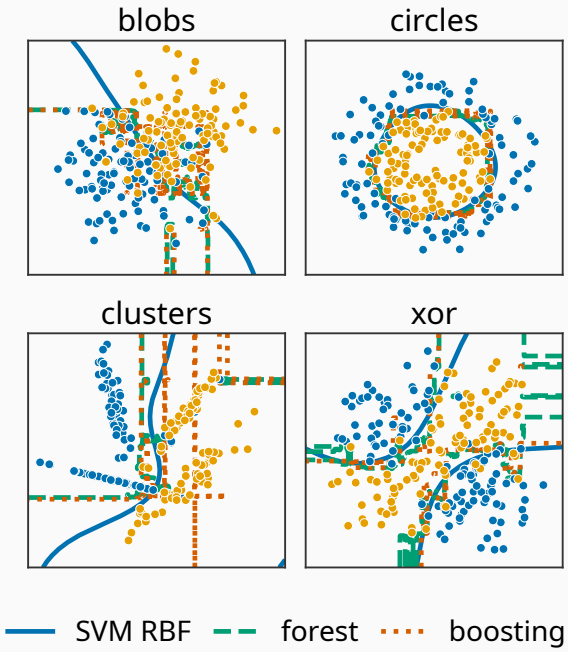
Each panel: the probability map of one model; filled points train, open points test.



- Straight, smooth, staircase and jagged boundaries on the same data

Reading the Gallery

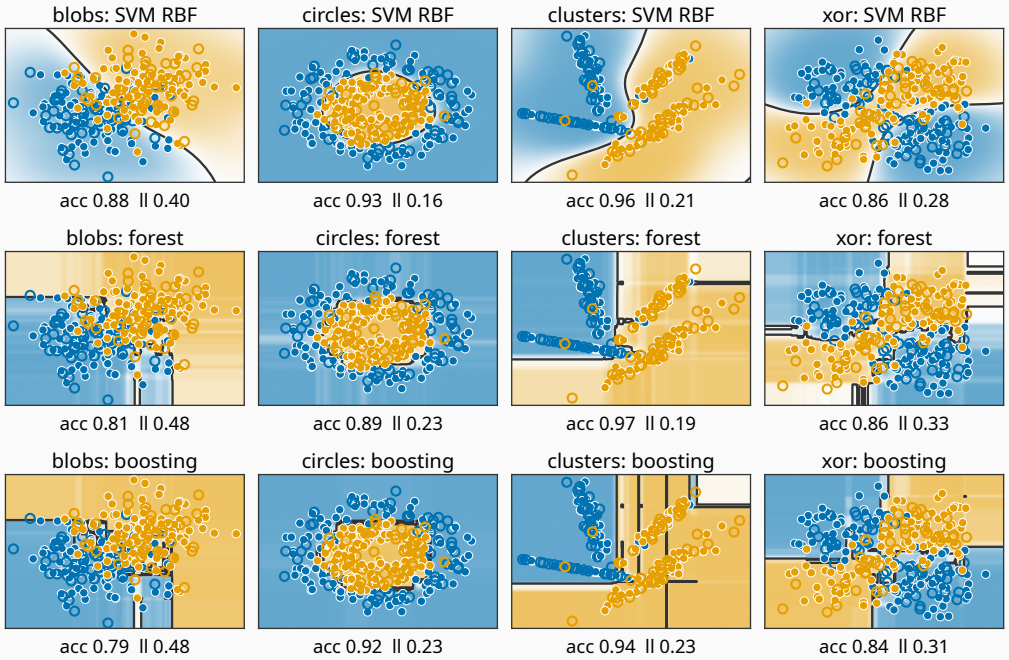
Three models on four sets, with the test accuracy under each panel:



- Moons test accuracy: logistic 0.756, SVM RBF 0.889, 15-NN 0.956, forest 0.900, boosting 0.922

Does the Ranking Hold?

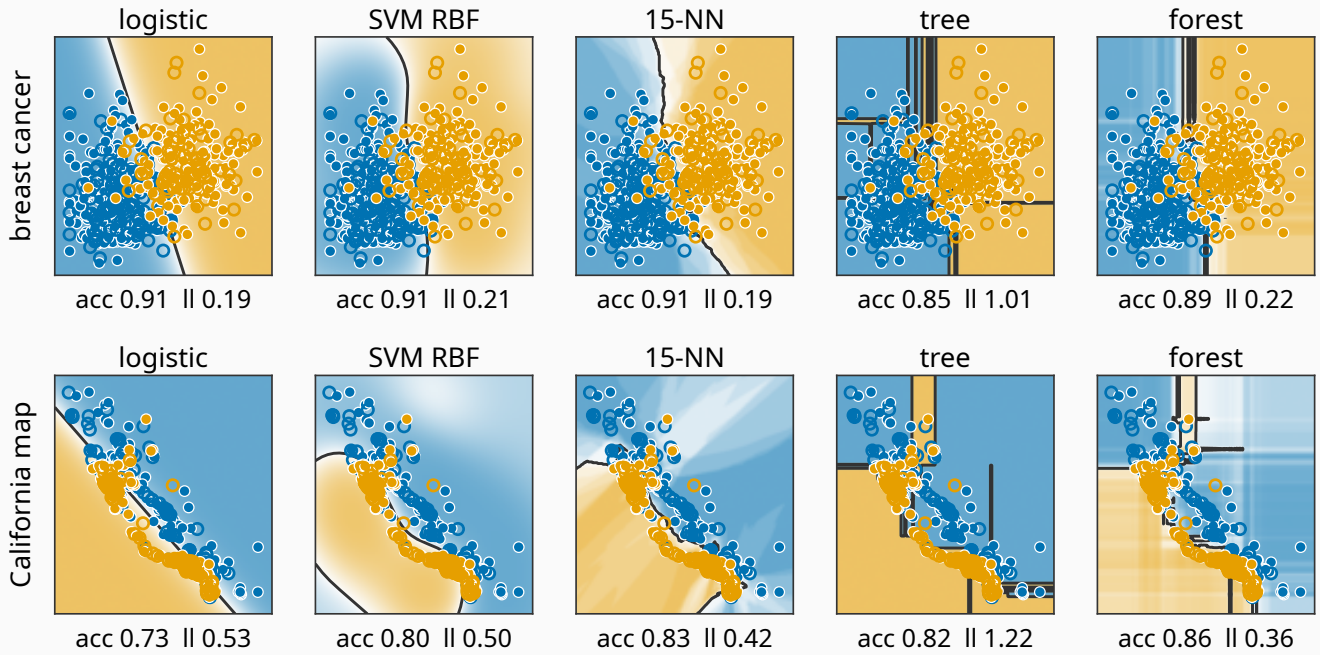
Five seeds on five sets: the best model changes with the set.



- Mean over 25 runs: SVM 0.901, MLP 0.900, 15-NN 0.897, tree 0.848, logistic 0.717
- Logistic regression fails on circles (0.46) and XOR (0.54): the bias of a line

Real Data in Two Dimensions

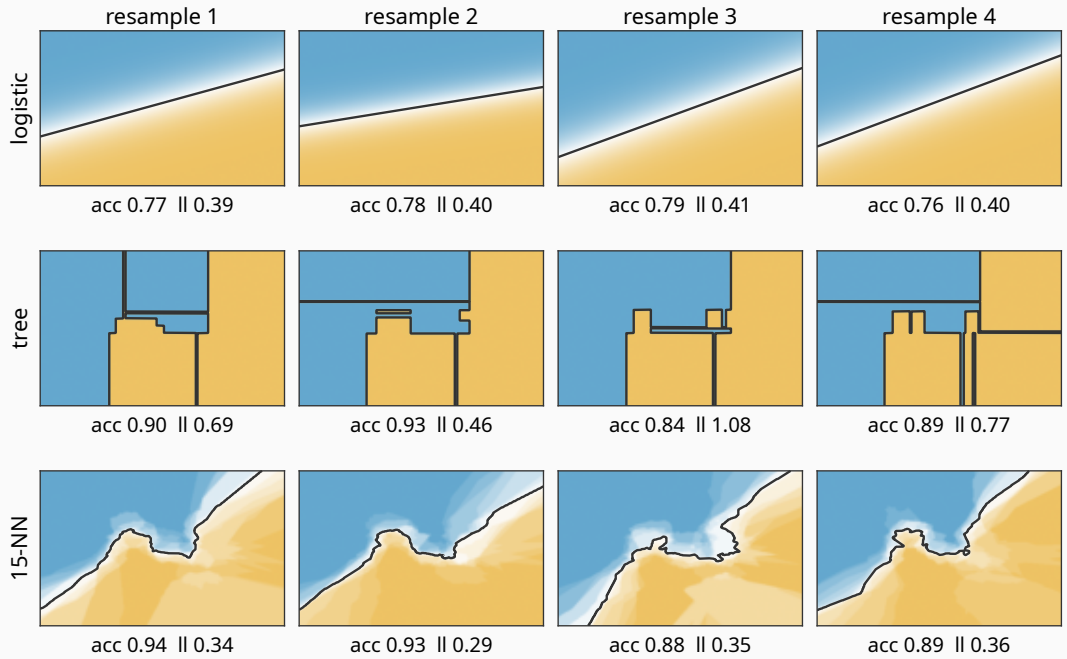
Two real tables reduced to two standardised features, so that a boundary can be drawn:



- Breast cancer (worst concave points, worst texture): logistic 0.912, 15-NN 0.906, tree 0.854
- California map (longitude, latitude; value above the median): logistic 0.733, forest 0.860

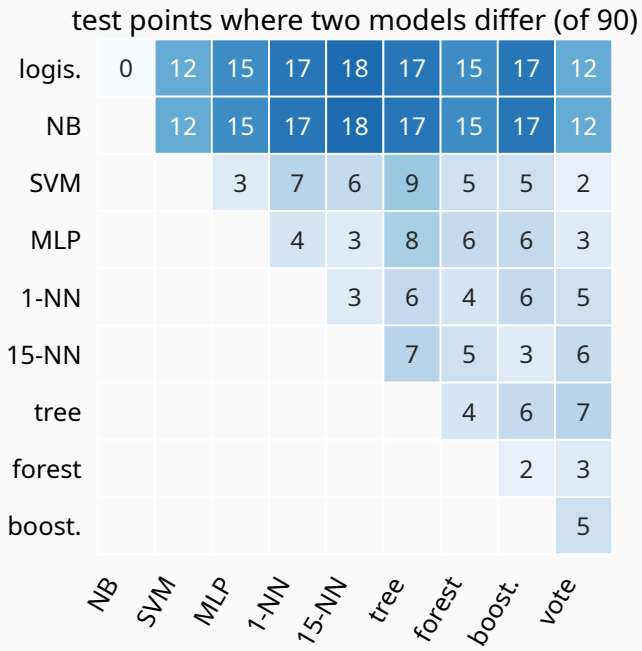
One Sample, One Model: Variance

Refit on four bootstrap resamples of the same 210 points:



- Logistic accuracy 0.772 (sd 0.014): stable, but biased
- Tree 0.892 (sd 0.037): the boundary moves with the sample
- **Variance** is what averaging removes (Class 01)

Do the Errors Overlap?



- 65 of 90 test points are right for all ten models, 3 wrong for all ten
- **22 points split the models**: that is room for a combination to help
- Oracle accuracy 0.967; majority of the ten hard votes 0.900

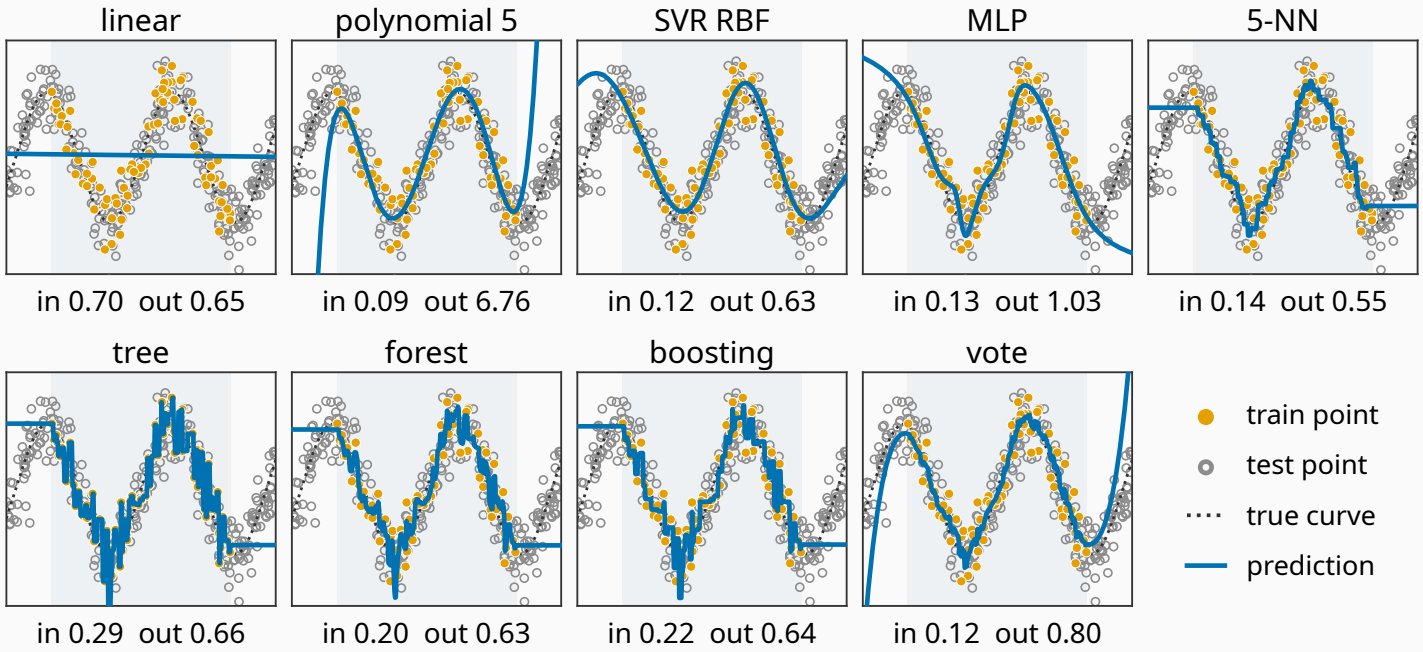
DEMO E1: Notebook

`01_decision_boundaries.ipynb` (E1) **shows** the gallery on five sets, the regression twin, disagreement

Look for: the noise sweep, the ceiling, real data in 1 and 2 dimensions, and the extrapolation panel

Regression Twin: Curves and Extrapolation

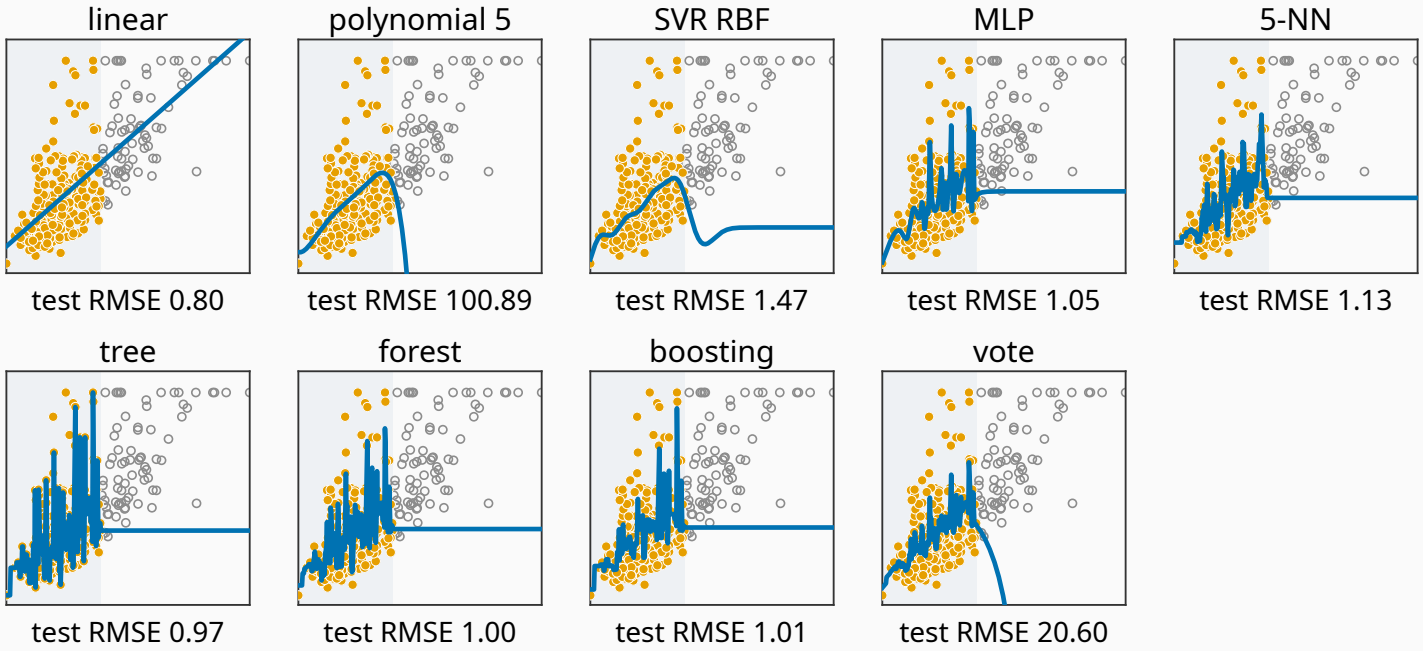
A noisy sine, trained on $[-3, 3]$ and queried on $[-4.5, 4.5]$: the shaded band is the training range.



- RMSE inside / outside: polynomial 0.092 / 6.764, SVR 0.120 / 0.634, forest 0.197 / 0.632
- Trees, forests and neighbours go flat outside the data; the polynomial explodes

Real Data: House Value against Income

California housing, trained on the districts up to the 75th income percentile, queried on all:



- RMSE on all districts: linear 0.804, tree 0.972, forest 1.002, polynomial 100.891
- A real target is noisy and capped: a line is hard to beat, a high-degree polynomial is not usable

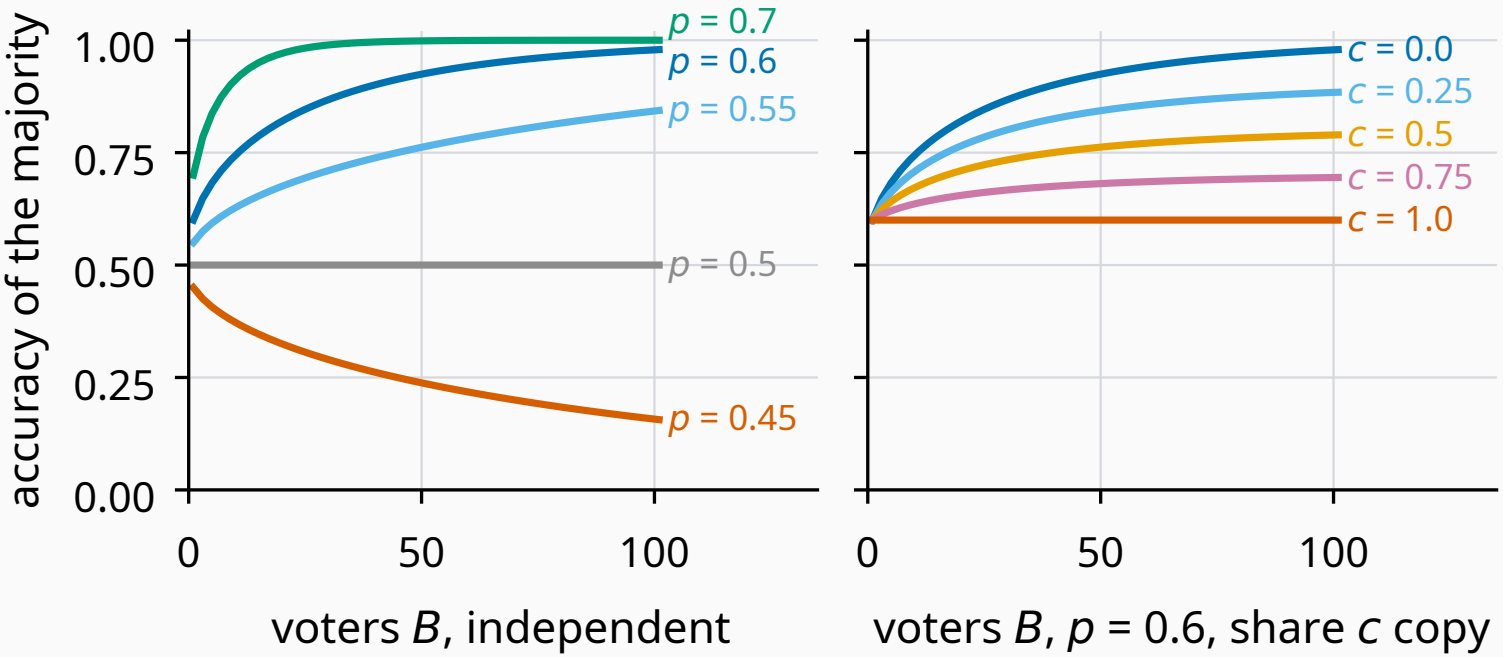
The Ceiling of the Moons

- A combination cannot beat the Bayes ceiling of about **0.92**
- The best single model (15-NN, 0.956) is lucky on 90 points
- So the honest claim is: *as good as the best member, better than the average member, with lower variance*
- Gains show against the tree and on real data (California housing, later)

Why Combining Helps

The Crowd Wins If Voters Are Independent

B voters, each right with probability γ; the majority is right with the binomial sum.



- $\gamma = 0.6$: $B = 3$ gives 0.648, $B = 11$ gives 0.753, $B = 101$ gives 0.979

Condorcet for Three Voters

$$
\begin{aligned}
a_3 &= \gamma^3 + 3\gamma^2(1-\gamma) && \text{(definition)} \\
&= 0.6^3 + 3 \cdot 0.6^2 \cdot 0.4 && \text{(substitute } \gamma = 0.6\text{)}
\end{aligned}
$$

- Three voters: the majority is 3 right or 2 right; $0.216 + 0.432 = 0.648$
- Assumption: the voters err **independently**
- Copies do not help: if half the voters copy, $B = 101$ gives 0.790; all copies stay at 0.600

When a Crowd Fails

- **Worse than chance:** $\gamma = 0.45$, $B = 101$ gives 0.156
- **Identical members:** three copies of 15-NN vote 0.956, like one
- **Shared bias:** three linear members on XOR vote 0.756, no better
- A useful member is better than chance **and** different from the others

Averaging Reduces Variance

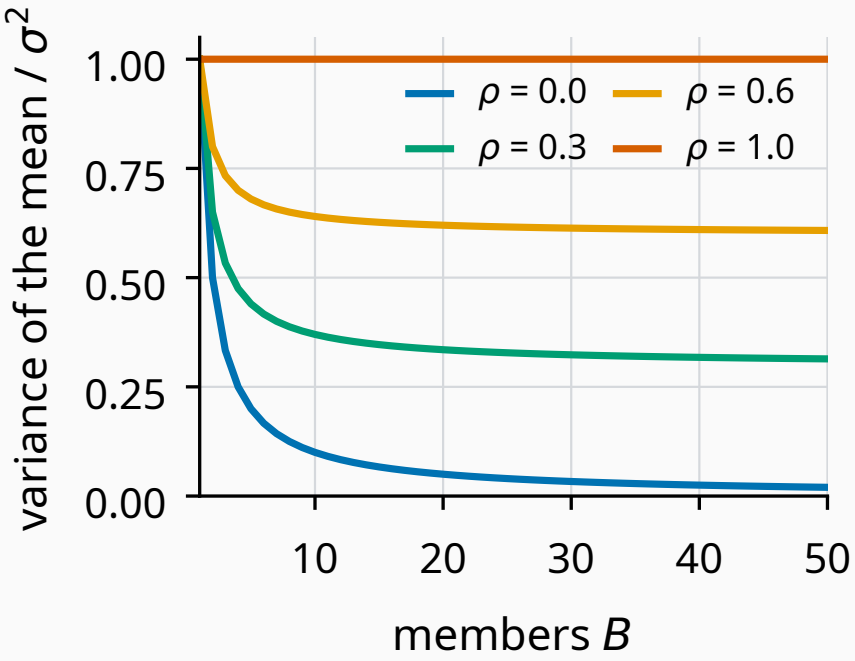
For regression: the mean of B predictions, each with variance v and pairwise correlation ρ:

$$V_B = \frac{B\,v + B(B-1)\,\rho\,v}{B^2} \quad \text{(definition)}$$

$$= \rho\,v + \frac{(1-\rho)\,v}{B} \quad \text{(simplify)}$$

- With $v = 1$, $\rho = 0.6$: $B = 5$ gives 0.680, $B = 50$ gives 0.608

The Floor Is the Correlation



- $\rho = 0$, $B = 50$: variance 0.020; $\rho = 0.3$: 0.314; $\rho = 0.6$: 0.608
- More members cannot remove the shared part ρv
- **Diverse members matter more than many members**

The Error of the Mean

Five members on the noisy sine: the mean is never worse than the average member.

- Mean member MSE 0.1084; spread of the members (ambiguity) 0.0153
- MSE of the mean $= 0.1084 - 0.0153 = 0.0931$ (the identity is exact)
- More **disagreement** among accurate members means a larger gain

Three Routes

Bagging, forest
parallel members
on resamples
cut *variance*

Boosting
members in sequence,
each fixes the last
cut *bias*

Stacking
a model
learns the mix
learn the combination

- The next parts take them in this order; a plain vote comes first

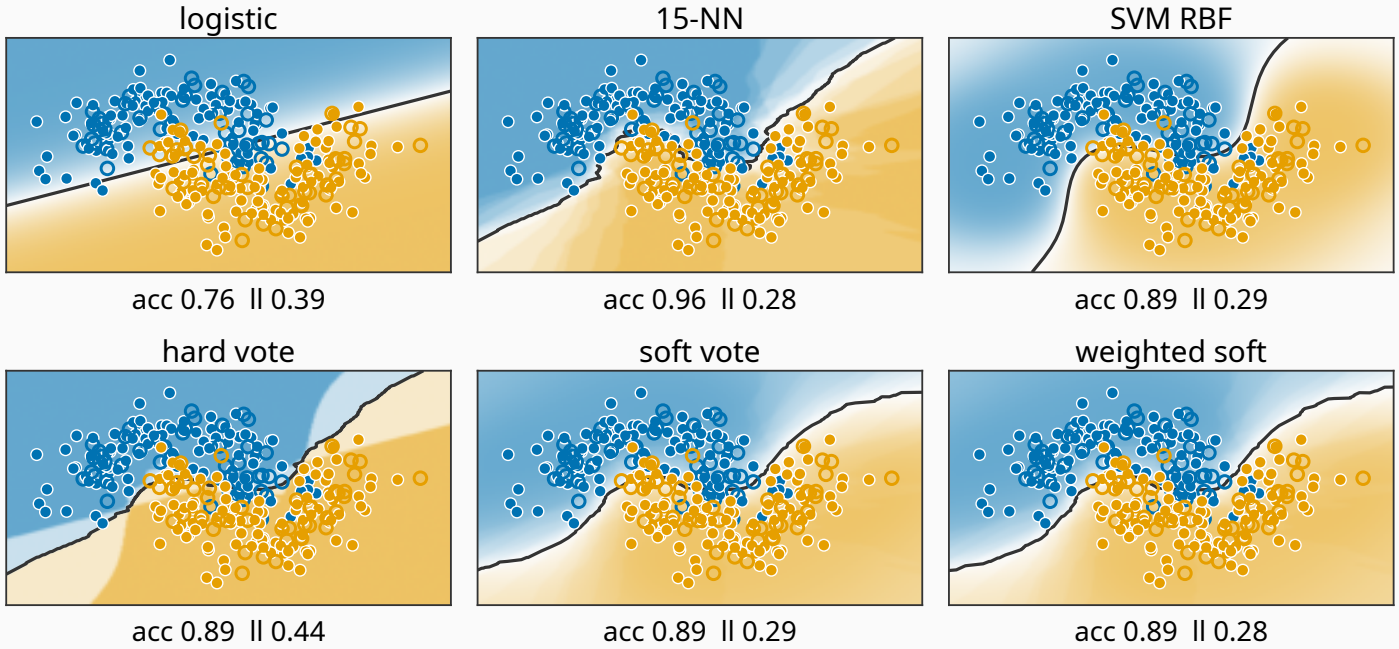
Voting

Hard Vote, Soft Vote

- **Hard vote:** each member names a class; the majority wins
- **Soft vote:** average the predicted probabilities, take the largest
- **Weighted:** members with a larger cross-validated accuracy count more
- For regression: the mean of the predictions

$$\hat{f}_{\text{ens}}(x) = \arg\max_{k} \frac{1}{B} \sum_{b} \hat{p}_b(k \mid x) \quad \text{(definition)}$$

Voting in Action



- Logistic 0.756, 15-NN 0.956, SVM 0.889: hard, soft and weighted vote all 0.889
- Log loss: hard 0.435, soft 0.288, weighted 0.284: **soft is better calibrated**

DEMO E2: Notebook

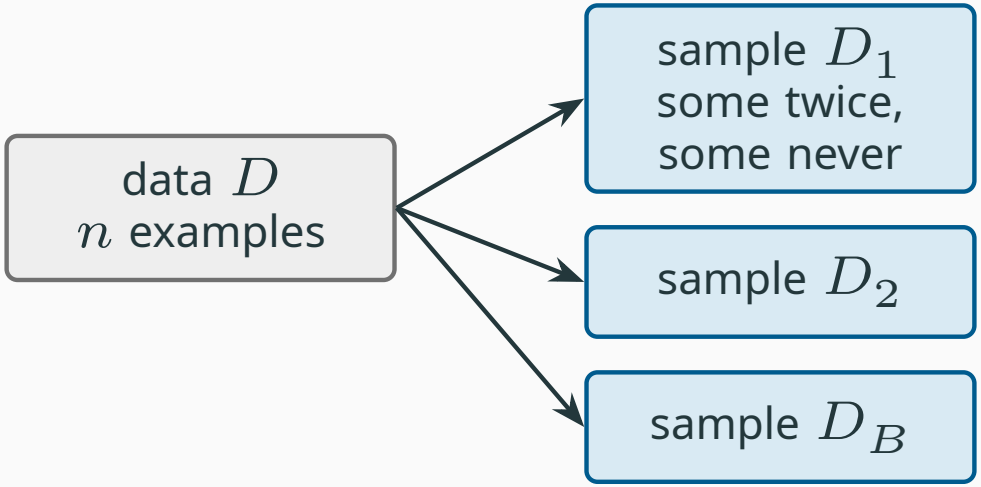
`02_bagging_forests.ipynb` (E2) **shows** hard, soft and weighted votes, and the failure cases

Look for: XOR, copies of one model, and ten seeds

Bagging and Random Forests

The Bootstrap Sample

To get B different training sets from one, draw n examples **with replacement**.



- Each sample holds about two thirds of the distinct examples (next frame)

How Many Distinct Examples?

$$\xi = \left(1 - \frac{1}{n}\right)^n \quad \text{(definition)}$$

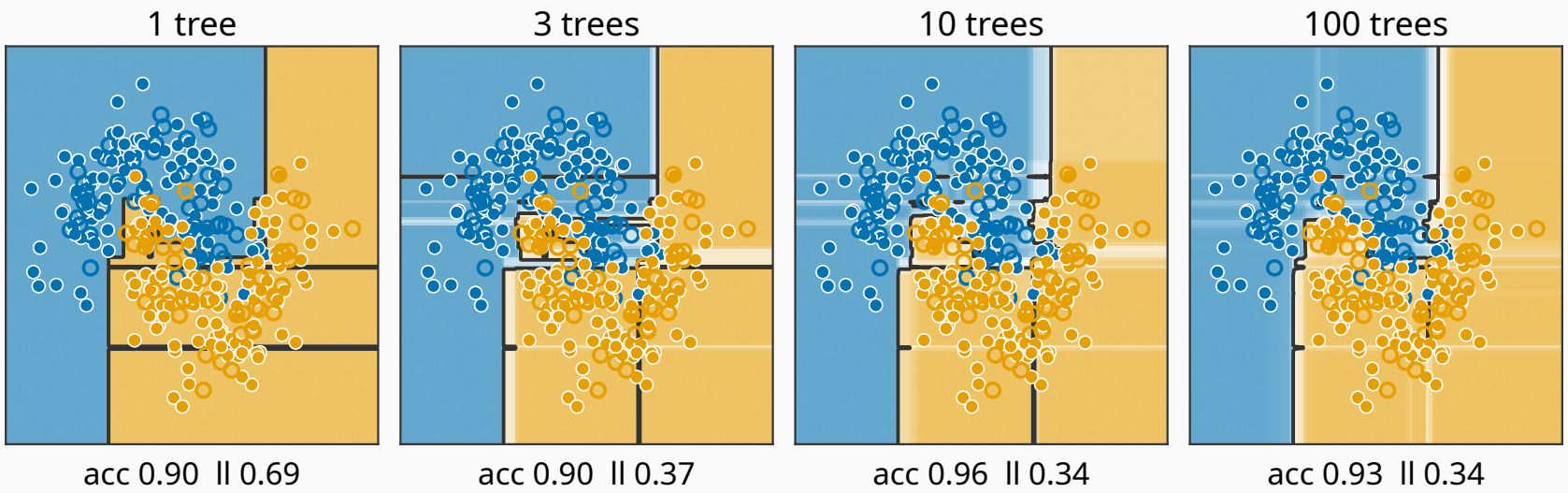
$$\xi \to e^{-1} \quad \text{(identity: limit)}$$

- ξ: all n draws miss example i; numerically $\xi = e^{-1} = 0.368$, so each example is present with probability $1 - \xi = 0.632$
- Check: unique share 0.657 ($n = 10$), 0.638 (50), 0.633 (210), 0.632 (10 000)
- The 0.368 left out is the **out-of-bag** (OOB) set of that member

Bagging in Five Lines

1. For $b = 1, \dots, B$: draw a bootstrap sample D_b from D
2. Fit a deep tree $\hat{f}_b$ on D_b
3. Classify by the majority vote of $\hat{f}_1, \dots, \hat{f}_B$ (regression: the mean)
4. A deep tree has low bias and high variance: the average keeps the first and cuts the second
5. Members are trained independently: they can run in parallel

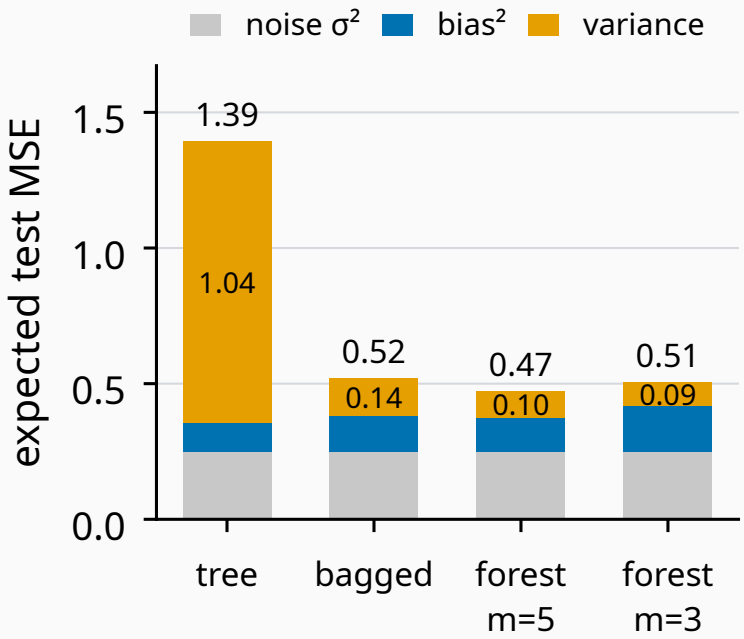
Bagging in Action: Trees Added One by One



- One tree 0.878; $B = 1, 3, 10, 100$: test 0.900, 0.900, 0.956, 0.933
- Log loss falls from 0.692 to 0.336: the boundary becomes smooth

Variance Cut, Bias Kept

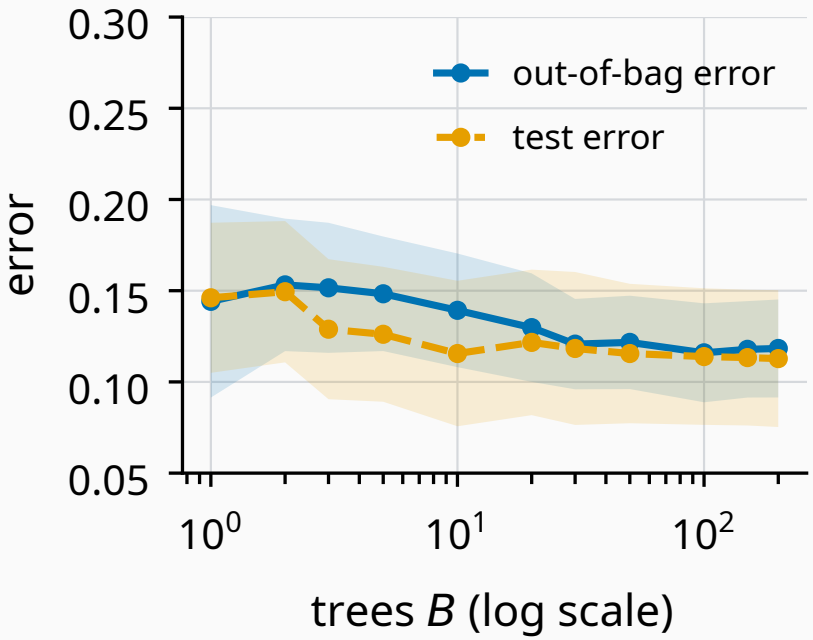
Bias and variance over 200 training sets (ten correlated features, noise 0.25):



- One tree: bias^2 0.105, variance 1.038, MSE 1.393
- Bagged: 0.131, 0.140, 0.521; forest ($m = 5$): 0.125, 0.097, 0.472
- Variance falls by a factor 7; the bias barely moves

Out-of-Bag Error: Free Validation

- Each tree never saw about 0.368 of the examples
- Predict each example with the trees that did **not** train on it

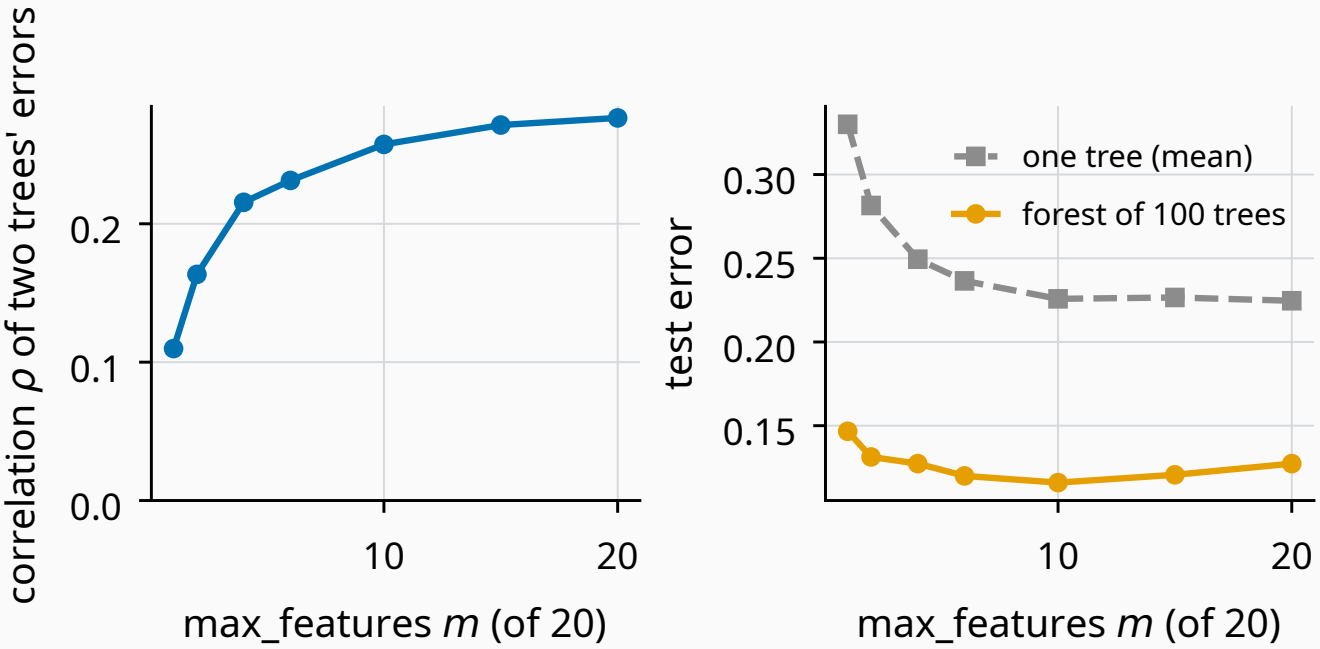


- Error at $B = 1, 10, 100$: OOB 0.144, 0.139, 0.116; test 0.146, 0.116, 0.114

From Bagging to a Random Forest

Bagged trees are still correlated: they all pick the same strong feature first.

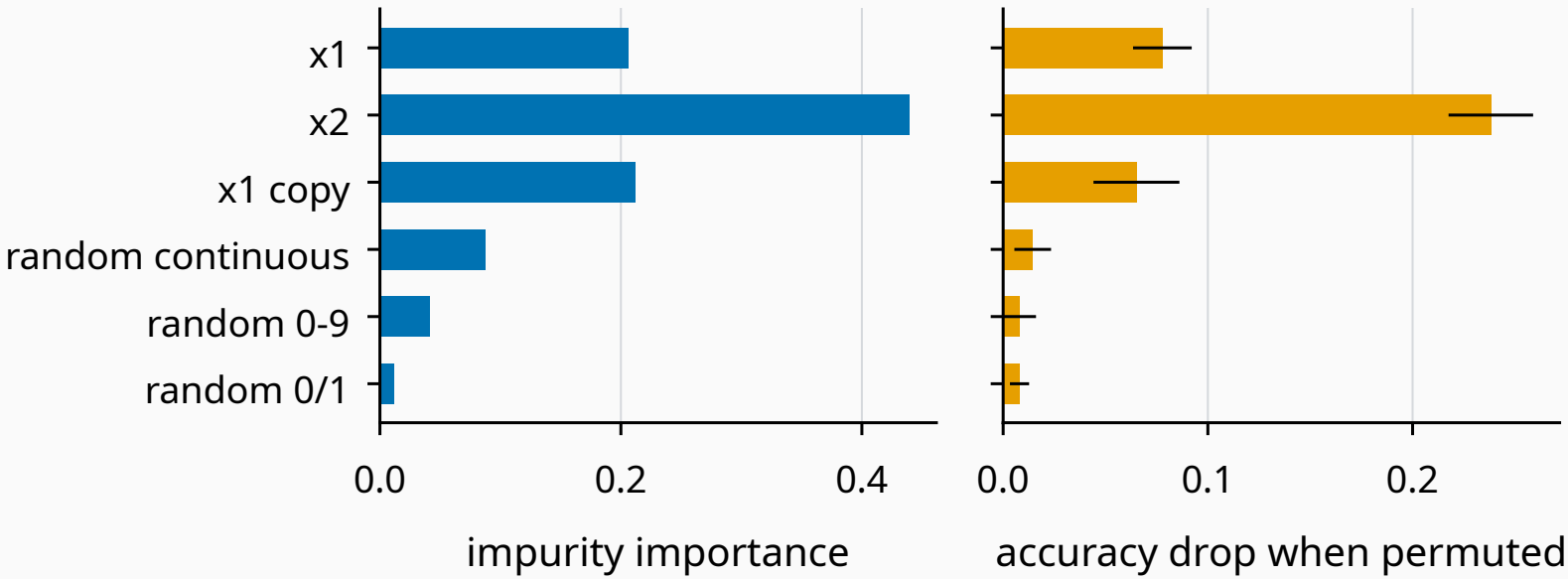
- **Random forest:** at every split consider only m of the d features, drawn at random
- Smaller m lowers ρ, so lowers the floor ρv; too small m makes each tree weak



The Few Hyperparameters

- **Members B:** more never hurts; stop when the OOB error is flat
- **`max_features`:** $\sqrt{d}$ for classification, a third of d for regression, then tune
- **`min_samples_leaf`:** larger leaves smooth the prediction
- Hand-made 100 trees: test 0.933 (OOB 0.933); `RandomForestClassifier` 0.900 (OOB 0.914)
- The two differ by chance on 90 points, not by a different method

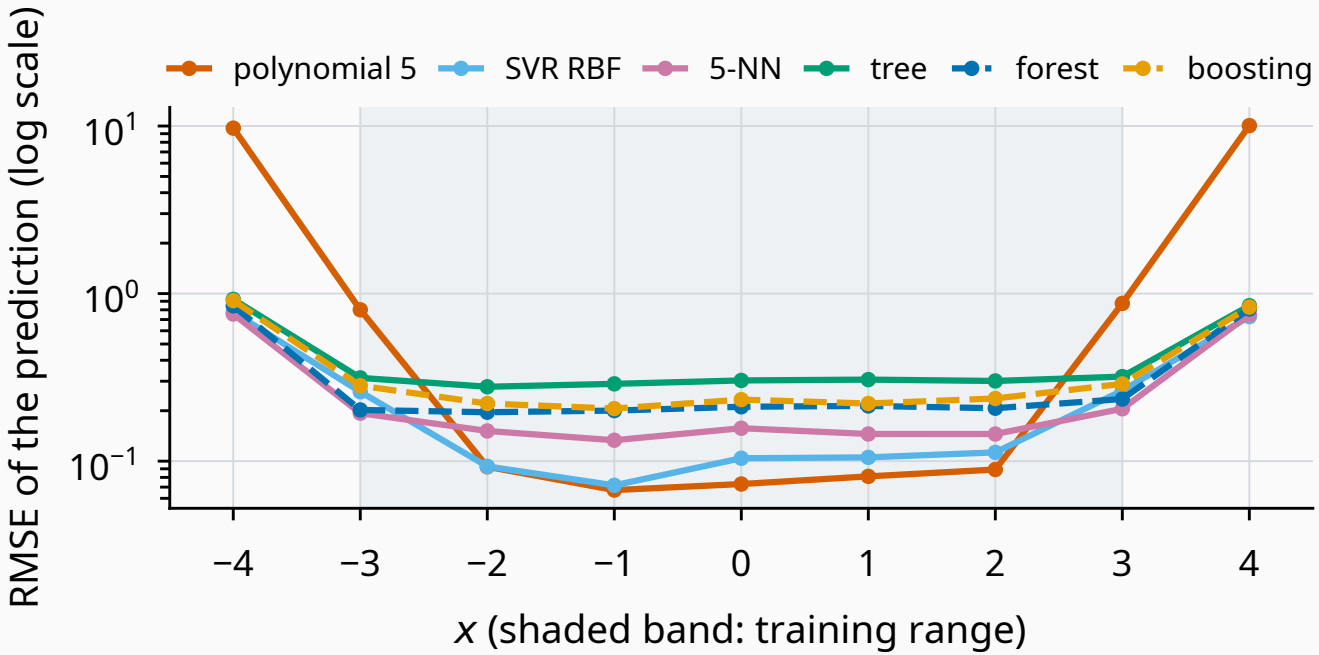
Feature Importance, Two Ways



- A random continuous column: impurity 0.088, permutation 0.014
- Noise share $14.1\,\%$ against $7.4\,\%$; a copy of a feature **splits** the credit

Limits of Bagging

- A **stable** learner gains nothing: 15-NN $0.902 \to 0.897$; a tree gains: $0.863 \to 0.883$
- A forest cannot extrapolate: outside the data it stays flat (Class 06)



DEMO E3: Notebook

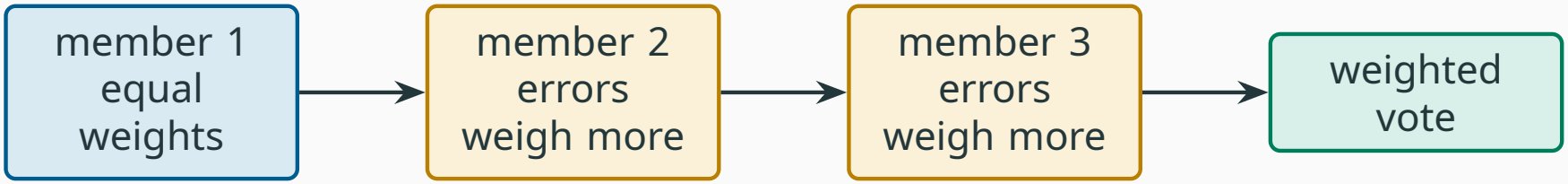
`02_bagging_forests.ipynb` (E3) **shows** bootstrap, OOB, ρ against m, bias and variance

Look for: `max_features`, 200 resamples, and bagging a stable learner

Boosting

The Idea: Fix the Mistakes in Turn

Train a weak model, find what it gets wrong, make those examples count more, train the next.



- The members are **stumps**: trees with one question (Class 06)
- Bagging cuts variance with deep trees; boosting cuts **bias** with shallow ones

AdaBoost on Ten Points: Round 1

Ten points of `make_moons(10, noise=0.2)`; every weight starts at $w_i^{(1)} = 1/10$.

- Stump $x_1 \le 0.053$ gets points 2 and 4 wrong: $\text{err}_1 = 0.2$
- Member weight $\beta_1 = 0.693$ (next two frames)
- Wrong points: weight $0.10 \to 0.25$; the others $0.10 \to 1/16$

The Member Weight

We minimise the loss E_b of adding member b with weight β_b (right: $e^{-\beta_b}$, wrong: e^{β_b}).

$$E_b = (1 - \mathrm{err}_b)\, e^{-\beta_b} + \mathrm{err}_b\, e^{\beta_b} \quad \text{(definition)}$$

$$e^{2\beta_b} = \frac{1 - \mathrm{err}_b}{\mathrm{err}_b} \quad \text{(set the derivative to zero)}$$

$$\beta_b = \tfrac{1}{2} \ln \frac{1 - \mathrm{err}_b}{\mathrm{err}_b} \quad \text{(solve for } \beta_b\text{)}$$

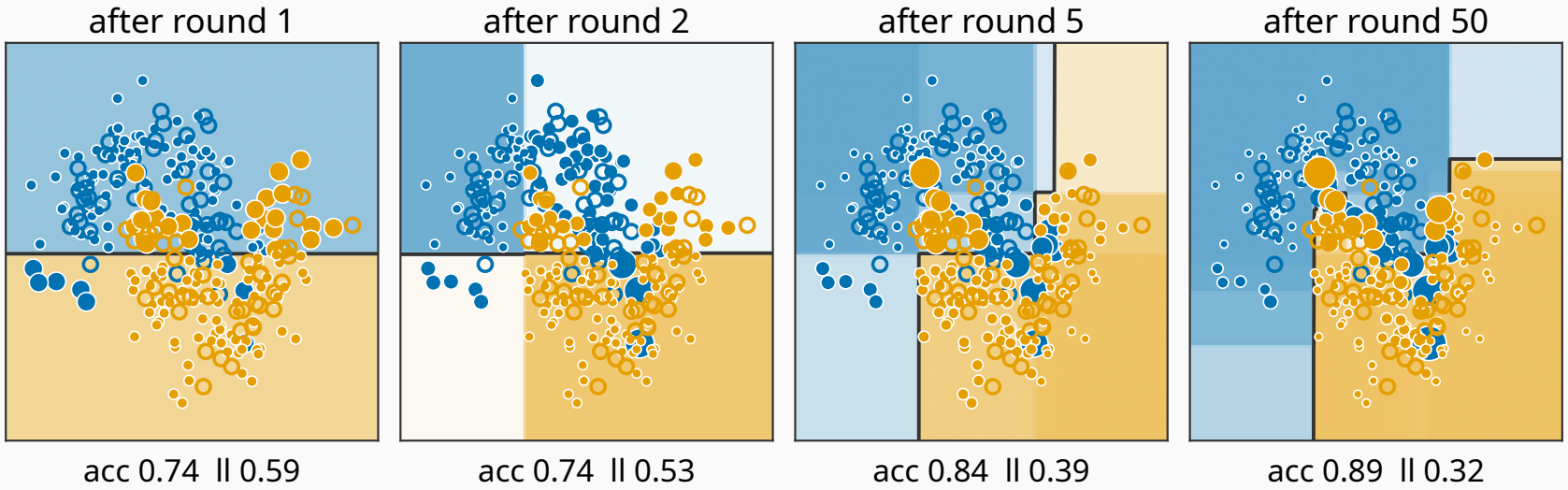
- $\mathrm{err}_1 = 0.2$: $\beta_1 = \frac{1}{2} \ln 4 = 0.693$; the loss falls to $E_1 = 0.800$
- `AdaBoostClassifier` reports $\ln \frac{1-\mathrm{err}}{\mathrm{err}}$: **twice** our β (1.609 against 0.805)

The Weight Update

$$w_i^{(b+1)} = \frac{w_i^{(b)}\, e^{-\beta_b\, y_i\, \hat{f}_b(x_i)}}{Z_b} \quad \text{(definition)}$$

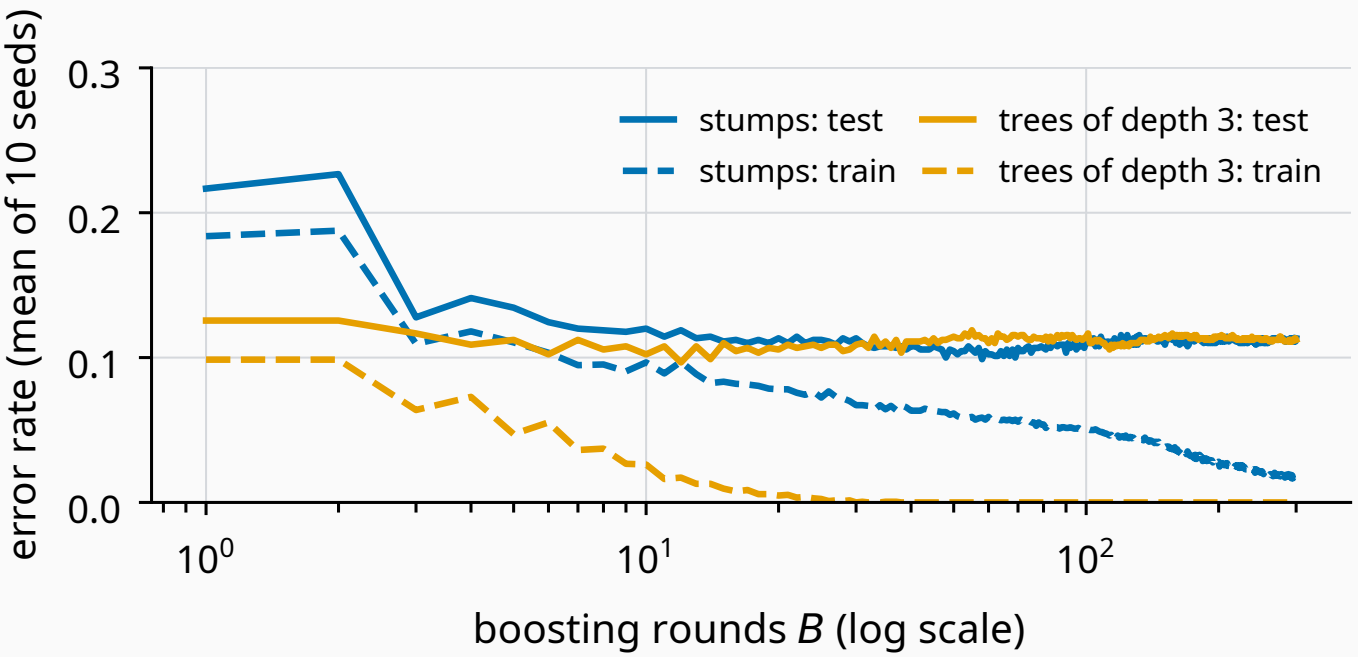
- Z_b makes the weights sum to 1; right points shrink, wrong points grow
- Labels $y_i, \hat{f}_b(x_i) \in \{-1, +1\}$; round 1: $Z = 0.800$, wrong $\to 0.25$, right $\to 1/16$
- Round 2: $\text{err}_2 = 0.125$, $\beta_2 = 0.973$; round 3: 0.214, 0.650
- The final vote sign $\sum_b \beta_b \hat{f}_b(x)$ is right on 10 of 10

AdaBoost in Action: Weights and Boundary



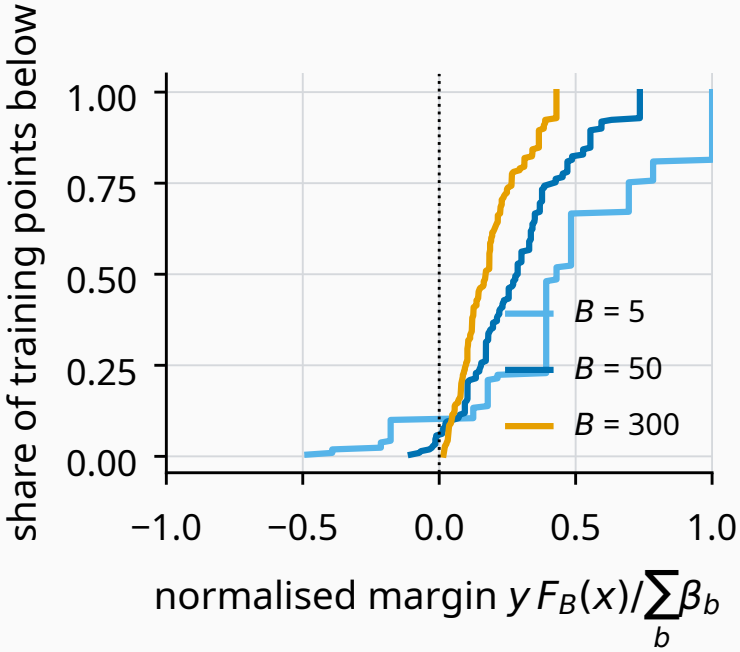
- After rounds 1, 2, 5, 50: test accuracy 0.744, 0.744, 0.844, 0.889
- The largest weight grows from $3\times$ to $13.8\times$ the starting $1/n$
- Hand-made and `AdaBoostClassifier` agree on every test point

More Rounds: When to Stop



- Stumps: best error 0.099 at 58 rounds, 0.113 at 300 (train error 0.018)
- Depth-3 members: best 0.097 at 12 rounds, then 0.114 with train error 0
- Boosting **can overfit**: the rounds are a hyperparameter

Margins Keep Growing



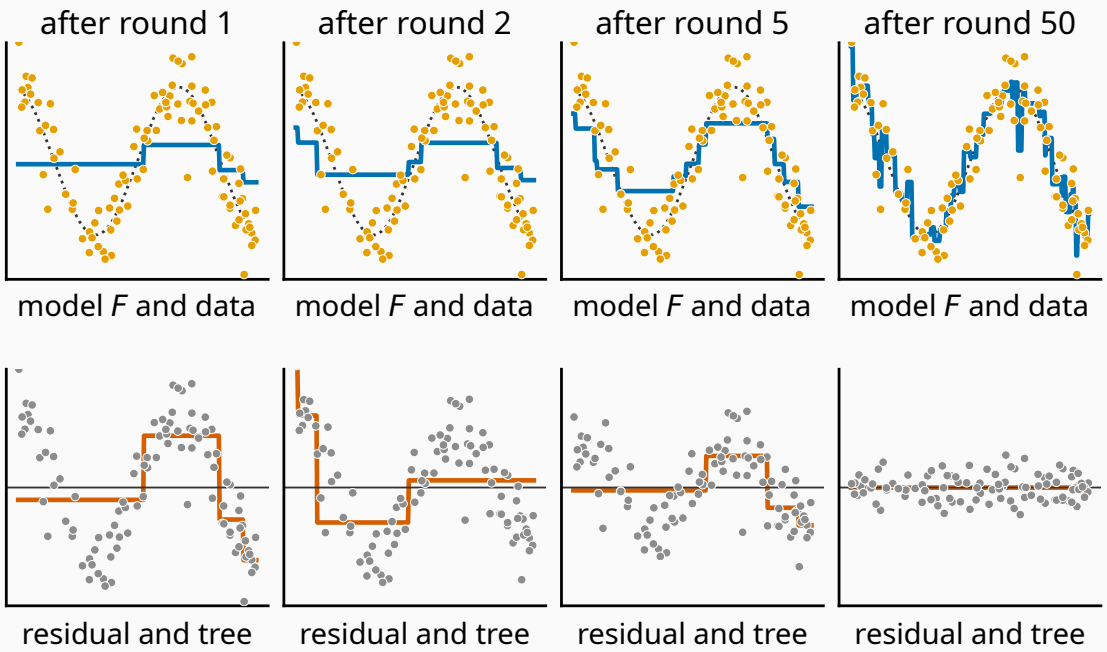
- Share of points with margin ≤ 0: 0.100 ($B = 5$), 0.057 (50), 0.000 (300)
- The vote gets more confident on the training points, even after the error is 0

Gradient Boosting: Fit the Residuals

Think of adding a member as a **step of gradient descent in the space of functions**.

- Start from a constant F_0 (the mean)
- Squared loss: the negative gradient is the residual $r_i = y_i - F_{b-1}(x_i)$
- Fit a small tree $\hat{f}_b$ to the residuals; set $F_b = F_{b-1} + \eta \hat{f}_b$
- Eight points, $\eta = 0.5$, stumps: MSE $0.461 \to 0.372 \to 0.337$

Residuals Shrink Round by Round



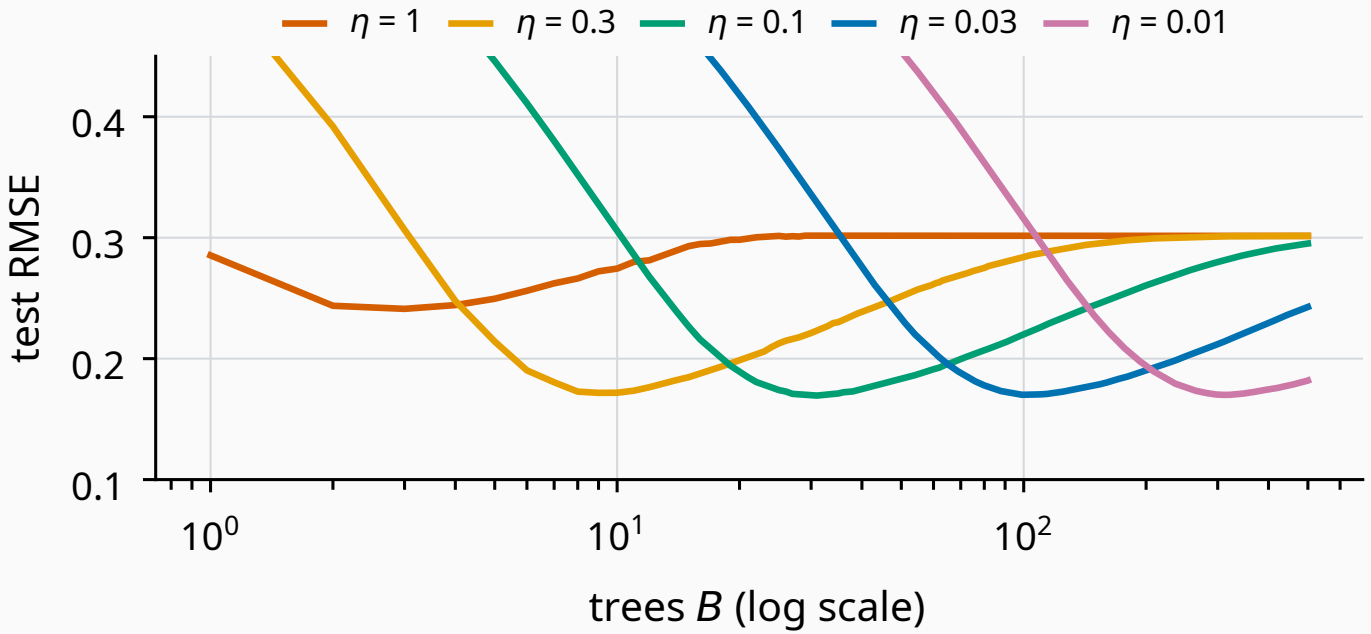
- 100 trees of depth 2, $\eta = 0.1$: test RMSE $0.708, 0.671, 0.645, 0.563, 0.448, 0.190, 0.172$ after $0, 1, 2, 5, 10, 30, 100$ rounds
- The hand-made version equals `GradientBoostingRegressor` to 10^{-15}

Another Loss, Another Residual

For the logistic loss the negative gradient is $y_i - \hat{p}_i$, the label minus the predicted probability.

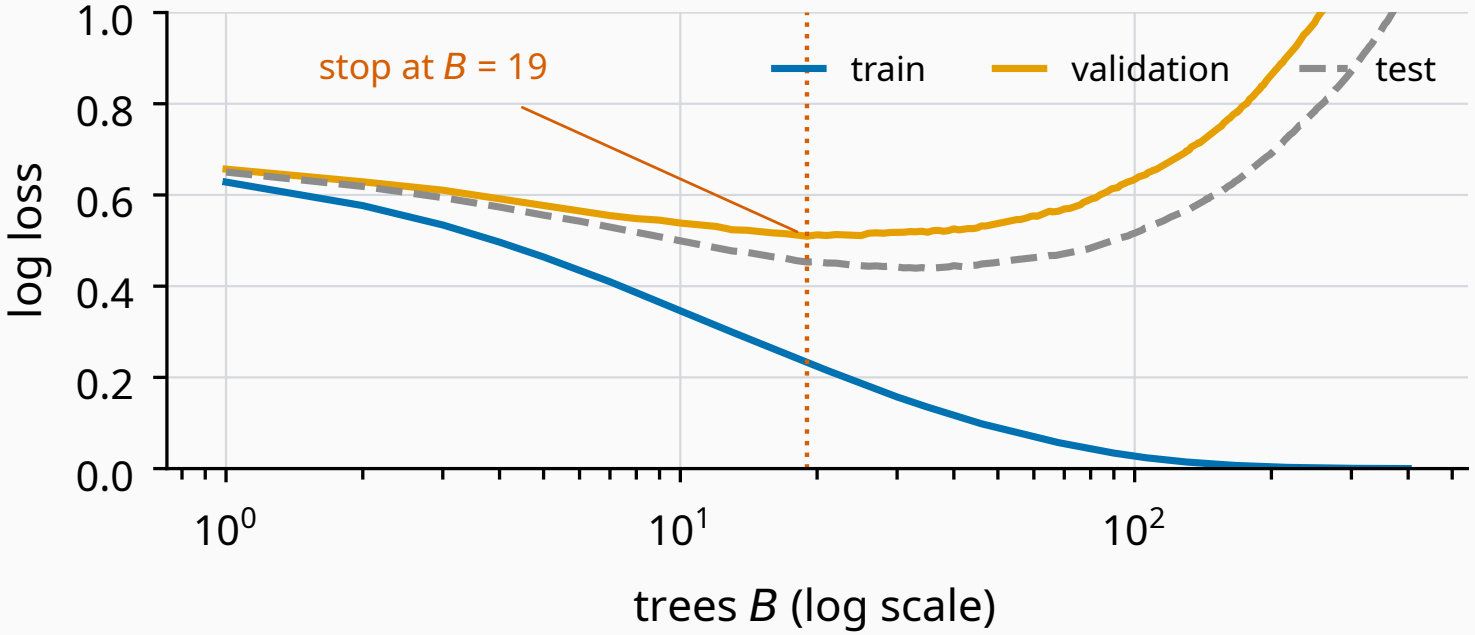
- Computed with `jax.grad` (Class 02): it matches $y - \hat{p}$ to 3×10^{-7}
- So one algorithm covers regression and classification: only the gradient changes
- This is the idea behind XGBoost, LightGBM, CatBoost

Shrinkage and Number of Trees



- $\eta = 1$: best 0.241 at 3 trees, 0.301 at 500 (overfit)
- $\eta = 0.1$: best 0.169 at 31 trees; $\eta = 0.01$: 0.170 at 315
- Smaller η needs more trees and gives a slightly better minimum

Early Stopping



- Heart disease, $\eta = 0.1$: validation log loss minimal at 19 trees (0.509)
- At 400 trees validation is 1.377 and test 1.057
- `n_iter_no_change = 10` stops at 32 trees on average; test accuracy 0.811 against 0.798

Fast Libraries

library	fit time	test RMSE
`GradientBoosting`	about 10 s	0.505
`HistGradientBoosting`	140 ms	0.499
XGBoost	90 ms	0.501
LightGBM	60 ms	0.505

- California housing, 16 512 rows, 200 trees; times are typical, not exact
- Histogram splits and leaf-wise growth make them 70 to 150 times faster
- CatBoost adds ordered target encoding of categories (Class 06)

Boosting Against Bagging

	bagging, forest	boosting
members	deep, in parallel	shallow, in sequence
cuts	variance	bias
label noise	robust	amplifies it
tuning	few knobs	rounds, η, depth

- Label noise 30 %: forest $0.947 \to 0.800$; AdaBoost stumps $0.948 \to 0.848$; depth 3 $0.943 \to 0.770$

DEMO E4: Notebook

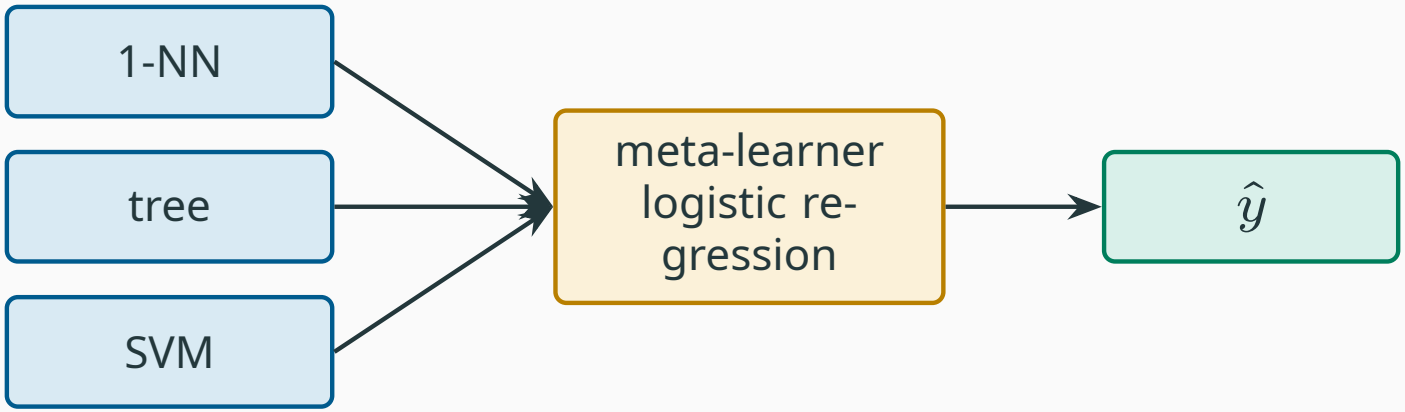
`03_boosting.ipynb` (E4) **shows** weights, margins, residuals, shrinkage, early stopping

Look for: the learning-rate grid, label noise, and fit times

Stacking

Learn the Combination

A **meta-learner** takes the members' predictions as its features.

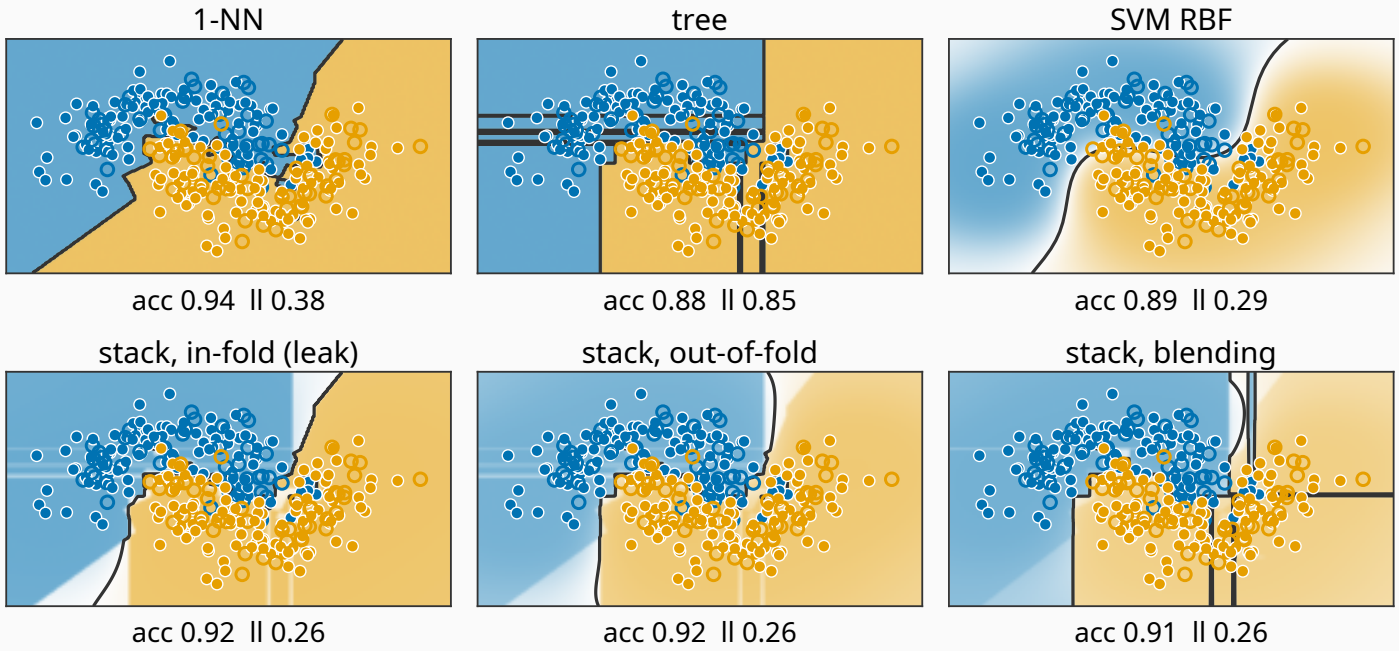


- The meta-learner can learn that the SVM deserves more trust than the tree

The Leakage Trap

- A member predicts its own training points too well: they were in its training set
- Feed those predictions to the meta-learner and it trusts the members too much
- **Out-of-fold predictions:** each example is predicted by a member that did not see it
- A **blend** uses one hold-out set instead

Stacking in Action: Leak or No Leak



- Meta-learner on in-fold features: train accuracy 1.000, 5-fold CV 1.000, test 0.922
- On out-of-fold features: 0.910, 0.914, 0.922: the honest estimate
- Ten seeds: leak 0.880, out-of-fold 0.888, blend 0.883

DEMO E5: Notebook

`04_stacking_and_choice.ipynb` (E5) **shows** the leak, the blend and the meta-learner strength

Look for: the weights of each meta-learner and a noise level of 0.4

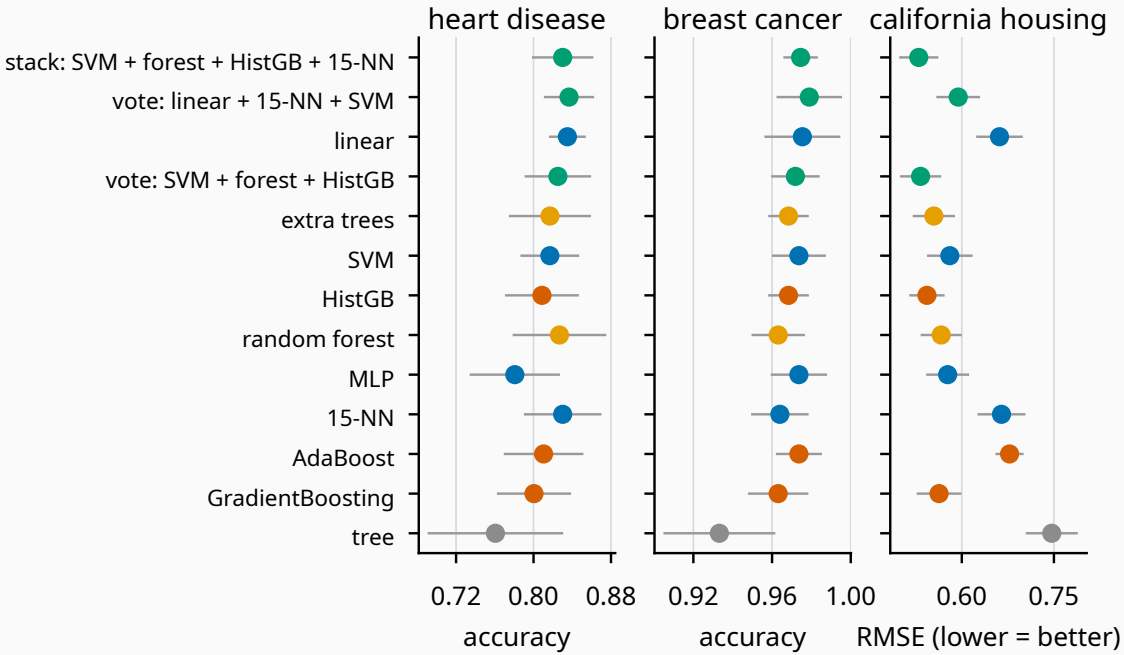
Using Ensembles

A Protocol on Real Data

Class 01 protocol: five-fold CV repeated twice ($J = 10$), corrected paired t-test against the best model.

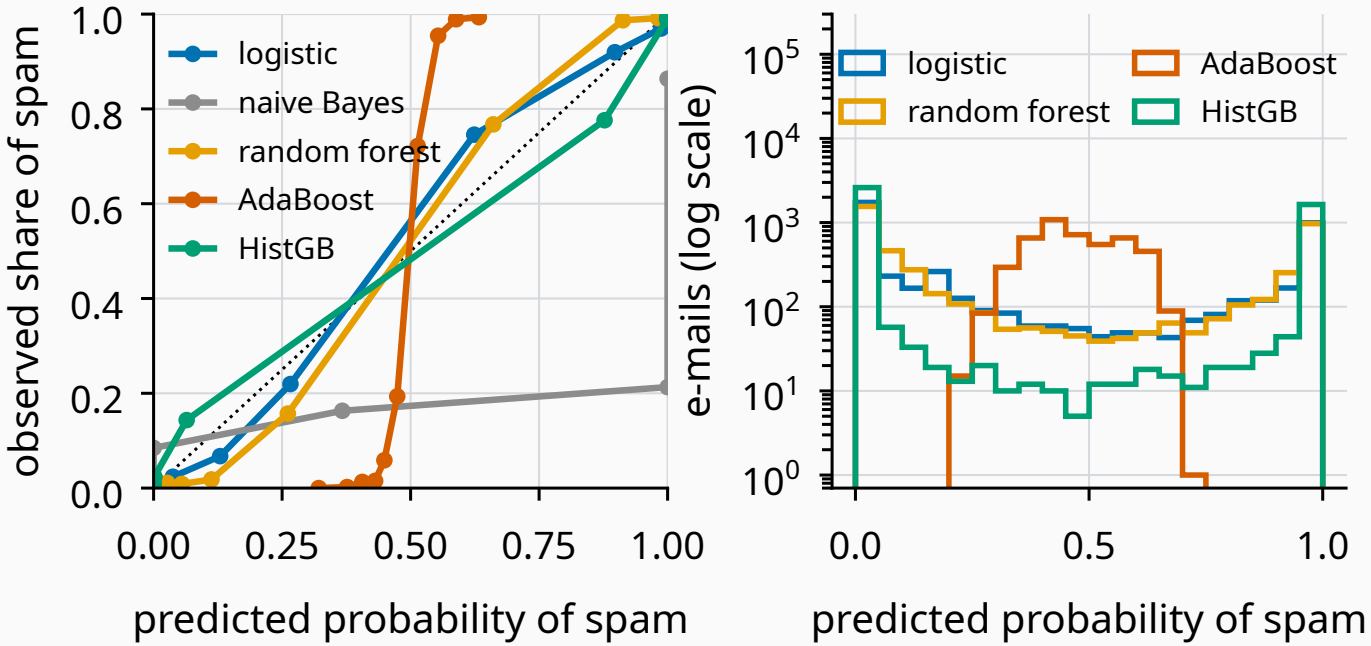
- Heart disease: vote 0.837, linear 0.835 ($p = 0.91$), forest 0.827, tree 0.761
- Breast cancer: vote 0.979, tree 0.933; HistGB against tree $+0.035$ ($p = 0.036$)

California Housing: Where Ensembles Win



- RMSE: stack 0.530, vote SVM + forest + HistGB 0.533, HistGB 0.543, forest 0.566, linear 0.661, tree 0.746
- Fit per fold (typical): stack about 7 s, forest 2.2 s, HistGB 0.7 s, linear 0.004 s
- HistGB against tree: -0.203 ($p < 0.001$); stack against HistGB: -0.013 ($p = 0.001$)

Calibration



- Spambase: log loss logistic 0.219, forest 0.151, HistGB 0.144, AdaBoost 0.549
- AdaBoost probabilities are poorly calibrated (gap 0.354): calibrate before using them

Which Ensemble, When?

	single model	forest	boosting	stack
accuracy	low	good	best	best
tuning effort	low	low	high	high
training cost	low	medium	medium	high
readable	yes	no	no	no

- A stack gains little over HistGB (0.013 RMSE) for 10 times the fit time
- Tabular data: start with a forest, then tune boosting

Ensembles and Project 1

- The symbolist (tree), the analogizer (KNN) and the connectionist (network) disagree in useful ways
- A soft vote or a stack over them is an ensemble of the tribes
- Check every claim with the Class 01 protocol, not one split

Summary

Summary: the Map

	bagging, forest	boosting	stacking
idea	average resamples	fix errors in turn	learn the mix
cuts	variance	bias	both
costs	memory, time	tuning, noise	time, leakage

Key Takeaways: Ensembles

- A crowd helps only if members are **better than chance and different**
- Averaging leaves a floor ρv: lower ρ with random feature subsets
- Bagging needs unstable learners; boosting needs a stopping rule
- Stacking needs out-of-fold predictions
- Near the Bayes ceiling no ensemble wins: say so, and test with J scores, not one split

Lab 07: Build It from Scratch

Part	Task	twin
A	Boundary gallery and a vote of three models	`VotingClassifier`
B	Bagging by hand: bootstrap, vote, OOB	`BaggingClassifier`, `RandomForestClassifier`
C	AdaBoost with stumps	`AdaBoostClassifier`
D	Gradient boosting, shrinkage, early stopping	`GradientBoostingRegressor`
E	Stacking with out-of-fold predictions	`StackingClassifier`
F	Challenge: pseudo-residuals with `jax.grad`	`HistGradientBoostingClassi`

- **In class:** Parts A to D (60 minutes); **at home:** Parts E and F
- Guide `practice/07_ensembles.pdf`; notebook `lab07_ensembles.ipynb`

References

Breiman, Leo. 1996. “Bagging Predictors.” Machine Learning 24 (2): 123–40.

Breiman, Leo. 2001. “Random Forests.” Machine Learning 45 (1): 5–32.

Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–94.

Dietterich, Thomas G. 2000. “Ensemble Methods in Machine Learning.” International Workshop on Multiple Classifier Systems, 1–15.

Freund, Yoav, and Robert E. Schapire. 1997. “A Decision-Theoretic Generalization of on-Line Learning and an Application to Boosting.” Journal of Computer and System Sciences 55 (1): 119–39.

Friedman, Jerome H. 2001. “Greedy Function Approximation: A Gradient Boosting Machine.” The Annals of Statistics 29 (5): 1189–232.

Grinsztajn, Léo, Edouard Oyallon, and Gaël Varoquaux. 2022. “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” Advances in Neural Information Processing Systems 35.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. The Elements of Statistical Learning. 2nd ed. Springer.

Ke, Guolin, Qi Meng, Thomas Finley, et al. 2017. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” Advances in Neural Information Processing Systems 30.

Wolpert, David H. 1992. “Stacked Generalization.” Neural Networks 5 (2): 241–59.